

Практическая работа №2

Дисциплина: «Инструменты девопс». ИПТИП, кафедра индустриального программирования.

Тема: Ветвление, Pull Request и командная работа в GitHub.

Выполнил: Алиев Никита, группа ЭФБО-01-24.

Преподаватель: Гиматдинов Дамир Маратович. Семестр 5, 2026/2027. Дата: 24.09.2026.

1. Цель работы

Освоить feature-ветки, GitHub Flow, Pull Request и review, работу с Git в IDE, сравнить Git и SVN, выполнить интерактивный rebase в отдельной ветке.

2. Создание feature-ветки

От main создана feature/hobby-project. В ней добавлен hobby.md с идеей, стеком, целью и критериями минимального результата системы консультаций. Создан коммит 07db4a6 с сообщением feat: add hobby project description.

```
$ git branch -a
* feature/hobby-project
  main
remotes/origin/HEAD -> origin/main
remotes/origin/main

$ git log --oneline --all --graph
* 07db4a6 feat: add hobby project description
* b94a6ba docs: add defense answers and course overview
* aaf7b43 docs: compare Git and SVN
* 70a3799 docs: add IDE vs CLI notes
* fcc4b62 docs: add semester goals
* 5a89cf5 Add about_me.md with student info
* 6b63fed Initial commit
```

Это фактический текстовый протокол, а не скриншот терминала.

Содержимое hobby.md

Мой пет-проект

Идея

Система управления учебными консультациями: студент выбирает преподавателя и свободное время, записывается на консультацию или отменяет запись. Преподаватель видит расписание и управляет доступными слотами.

Стек

Python 3.12+, Django, PostgreSQL, pytest, Docker и GitHub Actions. Основа — существующий учебный проект `consult_managing` с консольным меню.

Цель

Научиться развивать приложение через feature-ветки и Pull Request, автоматически проверять тесты в CI и запускать приложение в Docker.

Минимальный результат

- Расписание преподавателя и запись студента на свободный слот.
- Запрет пересечений записей и возможность отмены.
- Тесты основных сценариев, запускаемые при каждом PR.
- Документированная сборка и запуск контейнера.

3. Pull Request и Code Review

Подготовлены ветка и описание PR: что сделано, зачем и чек-лист. Отправка ветки и создание PR отклонены автоматической проверкой разрешений: публичная публикация персональных сведений требует отдельного подтверждения. Ссылка на PR отсутствует, поскольку PR ещё не создан. Парное review не выполнено: другой студент не назначен и его комментарии нет. Squash and merge и удаление feature-ветки должны выполняться после настоящего review и исправления замечаний. Чужие комментарии и одобрение не имитировались.

4. Работа с VS Code

Установлен GitLens 19.2.0. Создан `ide_notes.md` и отдельный CLI-коммит `70a3799`. Ручной коммит в UI и скриншот Source Control с diff остаются для демонстрации; автоматические команды терминала не подтверждают эти действия.

Git в VS Code и терминале

Три удобные возможности VS Code

1. Source Control (Ctrl+Shift+G) показывает изменённые файлы и позволяет выбирать отдельные изменения для индекса, не набирая пути вручную.
2. Встроенный diff показывает старый и новый текст рядом: легче заметить лишние изменения перед коммитом.
3. Редактор конфликтов показывает входящие и текущие изменения и помогает собрать итоговую версию. GitLens дополнительно показывает автора строки.

Что удобнее в терминале

Интерактивный rebase: `git rebase -i HEAD~3` позволяет явно задать порядок коммитов и squash. Эти команды можно повторить на сервере без графического IDE.

Демонстрация в интерфейсе

Этот файл создан и закоммичен через терминал по шагу 3.5 методички. Практику ручного коммита кнопкой  и личные впечатления необходимо подтвердить при демонстрации студентом;

автоматическое выполнение команд их не заменяет.

5. Сравнение с SVN

Создан отдельный коммит aaf7b43 с письменными ответами:

Сравнение Git и SVN

1. Почему ветвление в Git называют дешёвым?

Ветка Git — небольшой указатель на коммит. Создание новой ветки локально не копирует проект и не требует обращения к серверу. Фраза «ветки SVN всегда дорогие» неточна: серверный `svn copy URL1 URL2` использует cheap copies, занимает постоянное время и не дублирует всё дерево. Копирование рабочей папки на клиенте может быть медленным. Отличие SVN — ветка представлена каталогом репозитория и её фиксация требует доступа к нему. Источник: Version Control with Subversion — Using Branches (<https://svnbook.red-bean.com/en/1.7/svn.branchmerge.using.html>).

2. Можно ли сделать коммит SVN без интернета?

Для `svn commit` нужен доступ к репозиторию. Если он на недоступном удалённом сервере, коммит невозможен; редактирование и часть локальных операций доступны. Сам интернет не обязателен: сервер может находиться в локальной сети, а репозиторий на том же компьютере доступен через `file://`. Git хранит репозиторий локально, поэтому позволяет коммитить офлайн.

3. Когда SVN может быть предпочтительнее?

В организации с работающей SVN-инфраструктурой и высокой стоимостью миграции; при необходимости централизованных прав доступа к отдельным каталогам; при работе с плохо сливаемыми бинарными файлами, когда нужен процесс с блокировками. Это зависит от требований команды: Git не делает SVN непригодным автоматически.

6. Домашнее задание: интерактивный rebase

В `practice/rebase-playground` последовательно добавлены заметки об индексе, ветках и объединении изменений. После трёх коммитов выполнен `git rebase -i HEAD~3`. В `todo` второй `pick` заменён на `squash`. Редактор `todo` автоматизирован скриптом; операцию переписывания истории выполнил сам Git. Итог: два коммита вместо трёх. Хеш дерева до и после одинаков, значит содержимое сохранено.

ДО: три коммита

43d7020 docs: add integration practice notes

a8641f9 docs: add branches practice notes

49f1bca docs: add staging practice notes

ПОСЛЕ `git rebase -i HEAD~3`: два коммита

185c040 docs: add integration practice notes

b5b0b35 docs: add staging practice notes

Проверено: дерево файлов не изменилось. Ветка не публиковалась.

Ветка не отправлялась на GitHub, как требует задание. Для защиты: `git switch practice/rebase-playground`; `git log --oneline main..HEAD`.

7. Контрольные вопросы и подготовка к занятию 3

1. Почему защищают main? Чтобы изменения проходили проверку до попадания в стабильную ветку. Прямые коммиты технически допустимы, но политика команды обычно требует PR. Отдельный коммит часто можно отменить через revert; невозможность отмены не является общим свойством Git.
2. GitHub Flow. Создать ветку от main, сделать коммиты, push, открыть PR, получить review, исправить замечания, выполнить merge, удалить feature-ветку.
3. PR и merge. PR — предложение изменения с обсуждением и проверками; merge — операция объединения истории или изменений. PR может завершиться без merge.
4. Code Review. Проверка изменения другим участником: поиск ошибок, оценка понятности и обмен знаниями. Авторское самопроверение не заменяет парное review.
5. Git и SVN. Git распределённый: локально есть история и коммиты. SVN централизованный: коммиты поступают в единый репозиторий.
6. feat:. Тип коммита Conventional Commits для новой функциональности.
7. Первый push. `git push -u origin feature/hobby-project` публикует ветку и задаёт upstream для следующих push/pull.
8. Зачем CLI? Он доступен на серверах и в CI, позволяет писать скрипты и точно управлять Git без зависимости от интерфейса редактора.

Merge или rebase?

Merge выбираю для объединения опубликованной работы команды: он сохраняет историю ветвления и не переписывает существующие коммиты. Rebase выбираю для своей неопубликованной ветки: переношу её на свежий main или объединяю мелкие коммиты. Rebase создаёт новые идентификаторы коммитов, поэтому переписывать общую историю без согласования нельзя.

В упражнении `practice/rebase-playground` три коммита объединены в два через настоящий `git rebase -i HEAD~3`. Ветка оставлена только локально по заданию.

Источники: GitHub Flow (<https://docs.github.com/en/get-started/using-github/github-flow>), Pull requests (<https://docs.github.com/en/pull-requests/get-started/about-pull-requests>), Squash and merge (<https://docs.github.com/en/pull-requests/reference/pull-request-merges>).

8. Вывод и недостающие свидетельства

Созданы ветки и Conventional Commit, подготовлены идея проекта, заметки об IDE, ответы о SVN и интерактивный rebase. До полной сдачи нужны публикация, реальное парное review, исправление, Squash and merge, удаление ветки и скриншоты. Подробный перечень находится в `remaining.md`. Протоколы содержат реальные результаты команд и не подменяют отсутствующие действия.